Body Tissues Classification

Author Student IDs: 540179745, 550460880, 550057510, 541012132

Introduction

Aim

This study aims to identify corresponding cell tissues from human tissue slice images. Cellular tissues were classified into 9 major categories; each category represents a kind of cell tissue of the human body. We make predictions by training multiple models and select the best model among them. We hope to use model prediction to replace the human eye in identifying body tissue types. To achieve a higher level of accuracy than human vision.

Importance

Dataset

More accurate predictions on these body tissues can help doctors better distinguish the internal environment of the human body. Doctors can better identify the distribution of body tissues and tumours. This information can help doctors effectively assess a patient's physical condition to develop better surgical plans and medications. This makes cancer treatment more stable and reduces the risk of misdiagnosis.

Algorithm comparison

In this project, we will choose three models for body tissues prediction. They are random forests, fully connected neural networks, convolutional neural networks. We predict that convolutional neural networks will have the best predictive performance because they can capture feature information about positional changes through the sliding kernel. We also hope to discover potential possibilities by comparing multiple models.

Data

Description and exploration

This dataset contains 9 types of human tissue: adipose, background, debris, lymphocyte, mucus, smooth muscle, normal colonic mucosa, stromal and tumour. The feature images are 28 * 28 * 3 cell tissue slices. There are 32000 samples in the dataset which offers an extensive diversity. We decided to focus on studying the colour and brightness distribution of images. This allows us to better identify the correlation between image features and tissue types.

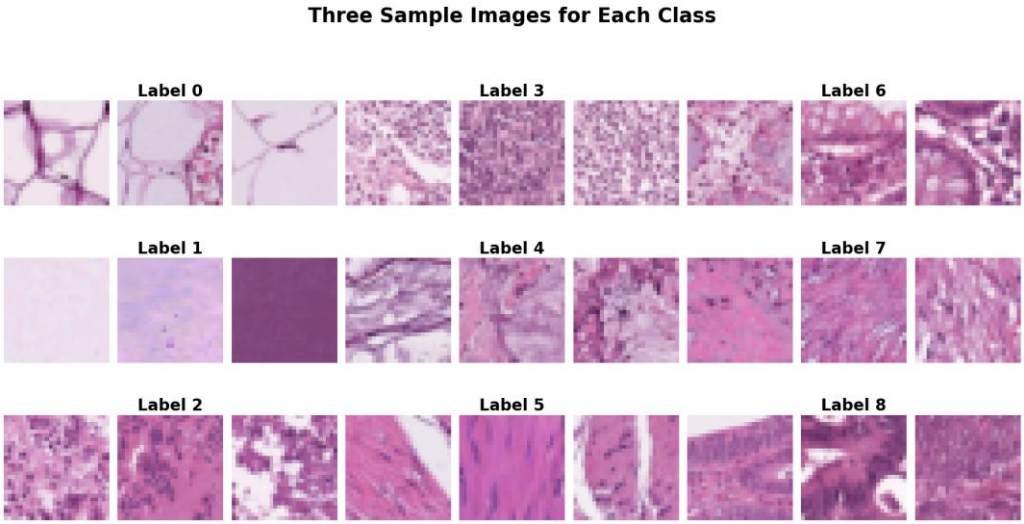

Figure 1. Sample Image of Dataset

The colour of the slices was determined by Hematoxylin and Eosin Staining. Hematoxylin binds to the acidic cell nucleus, turning it purple. Eosin binds to alkaline cytoplasm, extracellular matrix and erythrocytes, turning them pink. Fat, cavities and secretions appear white. The colour features of the image have great importance and need to be preserved. However, the brightness of images seems different. We decided to observe the different species by drawing diagrams.

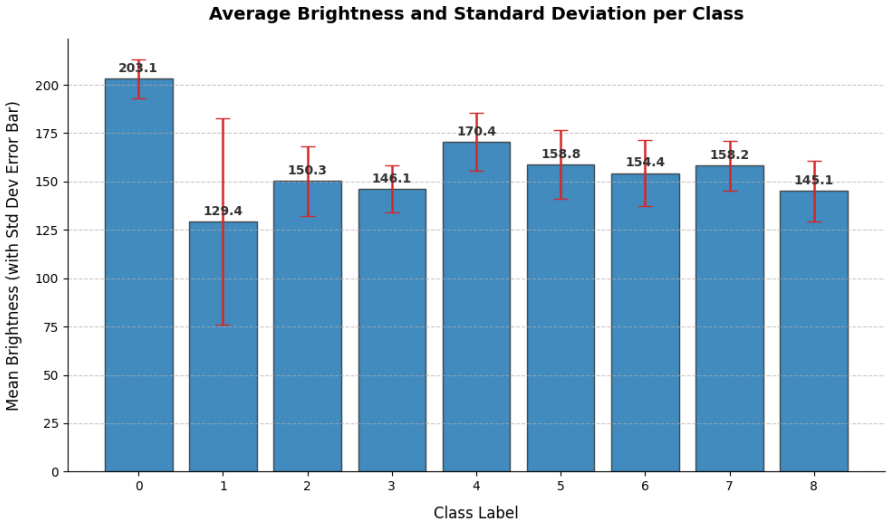

Figure 2. Average Brightness and Standard Deviation per Class

The distribution brightness of images seems different. This may be caused by using different experimental instruments. For example, adipose tissue sections have a lot of transparent white parts, which causes significant reflection during scanning, resulting in a brighter image. The model might learn this type of information to improve accuracy, but it doesn't belong to the characteristics of cellular tissue. The accuracy improvement in this way is not what we expect. We hope to mitigate this type of information by adding random brightness variations.

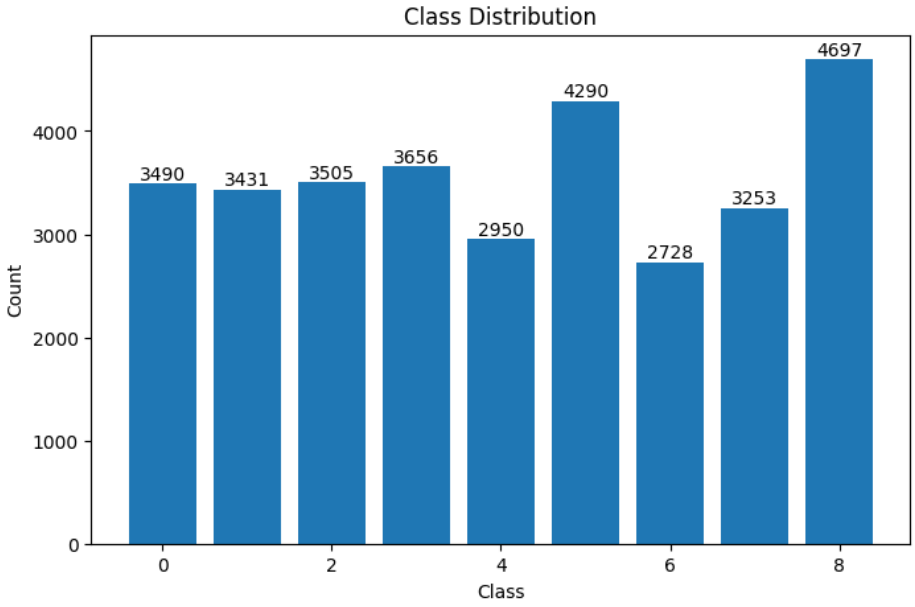

Figure 3. Number of Items in Each Class

The target variable for this dataset is a set of ordinal numbers. Each serial number corresponds to a type of body tissue. We plot the distribution of samples corresponding to each label by a histogram. The weight of each body tissue is well balanced in the dataset. The number of samples from the category with the most samples was 1.72 times that of the category with the fewest samples. The models we selected are robust to noise. For a slightly imbalanced dataset, no single class will dominate. We decided not to rebalance and to let the model maintain its natural distribution.

Pre-processing

First, we set the random seed to 0. The original dataset is large and relatively uniform. Therefore, we don't need to apply a rebalance on the training dataset.

For random forest, we apply:

- Flatten train and test into a one-dimensional array.
- Normalization.
- Dimensional reduction on image to 20 dimensions by PCA.

These operations ensure that it can complete the process within an acceptable time. We don't add additional transformation operations on random forest dataset because of the limitation of training time. The neuron network including MLP and CNN can speed up by GPU. This gives us the opportunity to apply transformation operations on it.

For neuron networks, we apply:

- Random rotate 0, 90, 180, 270 degrees when loading training image.
- Shifted the brightness and contrast when loading training image
- Shuffle the training image order in every epoch

These steps increase the sample diversity by adding random transformation. In addition, it reduces risk of overfitting on noise features such as direction, brightness and order for neuron networks.

Methods

Theory

Random Forest

Random Forest is an ensemble learning method based on decision trees. It builds many decision tree models to make predictions. Each tree model is built by a randomly chosen feature subset from the original training data with bagging. Each individual decision tree influences the final prediction result. Their influence on the prediction result is determined by their accuracy.

Fully Connected Neural Network

Multi-Layer Perceptron is a basic Feedforward Neural Network. MLP learns the complex relationship between input and output by using multiple layers of neurons and the activation function of each layer of neurons. The model includes input layer, hidden layer and output layer. Input layer is used to receive the feature data but no calculation. The hidden layer is used for weighted calculations and to apply the activation functions. Output layer is used to output the result. It updates the weights using backpropagation and gradient descent.

Convolutional Neural Network

Convolutional Neural Network is a neural network model specifically designed for processing image data. It extracts image features by setting a local receptive field through convolution operations, including convolution layer, activation function, pooling layer and fully connected layer. In convolution layers, the kernel slides across the image and learns the different features. Pooling layers can be used to reduce dimensionality.

Strengths And Weaknesses

Performance

CNN (Strong):

- Discover features at any location using moving convolutional kernels.
- Effectively identify the spatial structure of high-dimensional spaces.

Random forest (Weak):

- Flattening into a one-dimensional space destroys the spatial features.
- Hard to recognize the image structure, such as cell density and texture.

MLP (Weak):

- Unable to recognize the movement of key features.
- Easy to learn noise which is not a part of image structure.

Overfitting

Random forest (Strong):

- Each subtree is built by different features and samples.
- Many tree models help generalization.

CNN (Normal):

- Noise can be filtered by learning the geometric structure of the image.
- Space noise can be filtered through transforms such as rotation.

MLP (Weak):

- Learn a decision boundary in units of pixels rather than image structure.
- Risk to treat noise in the image as a feature for learning.

Runtime

Random forest (Strong):

- Training time is acceptable; low computation load for independent trees.
- Prediction time is acceptable; prediction results are voted by all trees.

MLP (Weak):

- Training time is long; backpropagation requires heavy computation.
- Prediction time is fast; prediction only requires matrix computation.

CNN (weak):

- Training time is long; it requires convolution, backpropagation.
- Prediction time is acceptable; complex computation on convolution.

Number of params

Random forest (Normal):

- Every tree has a group of parameters.
- Each tree has their own feature index and threshold value.

CNN (Normal):

- Weight and bias belong to kernels but not the whole image.
- Still have fully connected layers.

MLP (Weak):

- Every link between neurons has its own weight.
- Large number of neurons in each layer.

Interpretability

Random forest (Strong):

- Each feature has their calculated importance
- Each branch has a threshold value.

CNN (Normal):

- Can create Feature visualization of kernels.
- Can use a heatmap to show each areas' effect on result.

MLP (Weak):

- Weight and bias don't have obvious biological significance.
- Can't give any prediction logic for human understanding

Architecture And Hyperparameters

Description

Random forest implements based on RandomForestClassifier in sklearn. The tuning operation is achieved by combining parameters using the GridSearchCV mechanism. In addition, we use a 5-fold cross validation to prevent overfitting. We choose 3 hyperparameters for tuning:

- **n_estimators (250, 500):** Number of decision trees in a random forest. More estimators improve model's generalization but increase the runtime.
- **max_depth (25, 50):** The maximum depth of each single decision tree. Deeper trees predict more detail but have higher risk of overfitting.
- **min_samples_split (2, 5):** The minimum of samples required for splitting. Lower limit predicts more detail but have higher risk of overfitting.

MLP's first layer is a flatten layer that converts the input images to 1 dimension array. Following three groups of layers, each group has a dense layer to capture the relationship between features, a batch normalization layer to improve model stability, an activation layer to reduce the linearity between layers and a dropout layer for prevent overfitting. After all, a SoftMax layer outputs the probability of each cluster. We choose 3 hyperparameters for tuning:

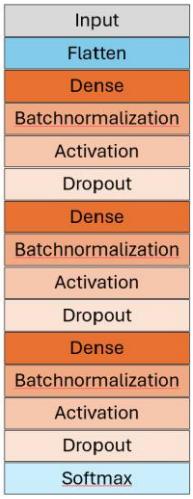

Figure 4. MLP Architecture

- **hidden_sizes ([128, 64, 32], [256, 128, 64]):** The number of neurons in the first three dense layers. More neurons make the prediction more detail but increase the runtime and risk of overfitting.

- **activation (sigmoid, relu):** The activation function after each of the first three dense layers. Sigmoid function reduces the gradient while relu function keeps the gradient.
- **lr (0.0001, 0.001):** The learning rate is the magnitude of information the model learns from each training session. Lower learning rate learning more detail but may find a local minimum instead of the global one.

CNN's first three layers are convolution layer, batch normalization layer and activation layer to capture image features, stabilize training and remove linearity. Following two basic blocks use skip connection method to prevent gradient vanishing. Next, use max pool to filter features with weak influence, and global average pool transform the output of the previous layer to 1 dimension. Then, a dropout layer is used to prevent overfitting. Finally output predicted probability by a SoftMax layer. We choose 3 hyperparameters for tuning:

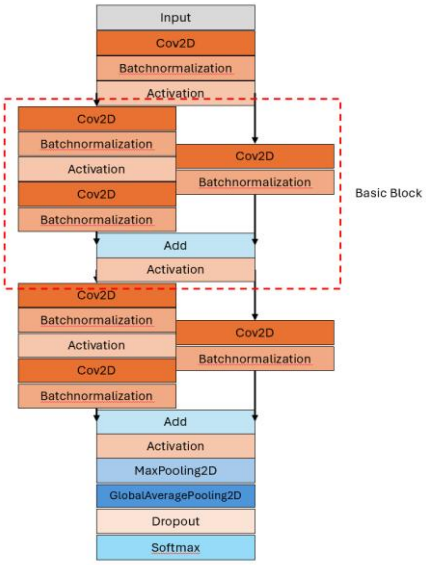

Figure 5. CNN Architecture

- **filter ([64, 128], [64, 128]):** Number of graphic features captured in the current layer. Models with more filters learn more graphic features but may overfit.
- **kernel size (3, 5):** The kernel size in the current layer. Larger kernel size has wider receptive field but may ignore details.
- **learning rate (0.0001, 0.001):** The learning rate is the magnitude of weight change in each training session. Lower learning rate learning more detail but may miss the global one.

Tuning Results Presentation

Random forest

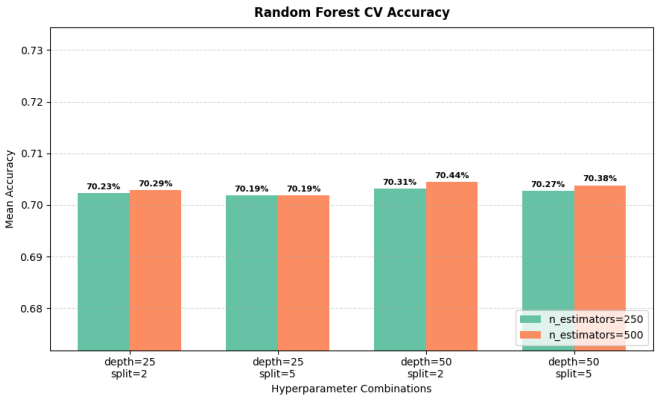

Figure 6. Random Forest Tuning Results

MLP

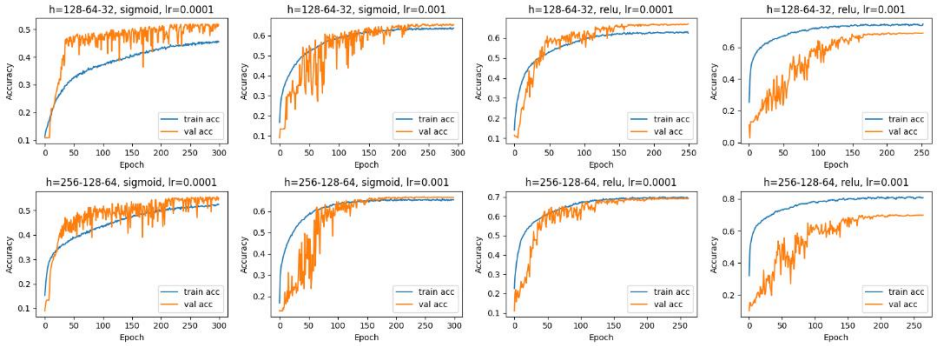

Figure 7. MLP Tuning Results

CNN

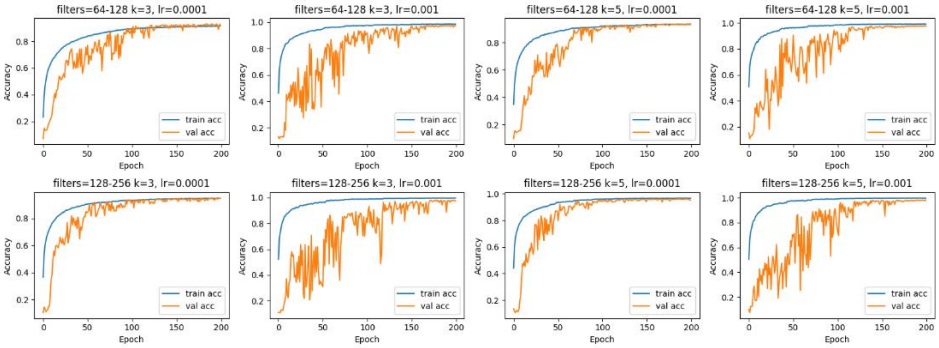

Figure 8. CNN Tuning Results

Tuning Analysis

Random forest

- 500 estimators have better performance than 250 estimators because it improves generalization.
- max depth 50 has better performance than max depth 25 because it learns more detail.
- Splitting with at least 2 samples has better performance than 5 samples because it learns more detail.

MLP

- 256, 128, 64 hidden layer size have better performance than 128, 64, 32 hidden layer size because it learns more detail.
- Relu activation function has better performance than sigmoid activation function because it keeps more gradient.
- 0.001 learning rate has better performance than 0.00001 because it is more likely to find global minima while using the learning rate reducer.

CNN

- 128, 256 filters have better performance than 64, 128 filters because it learns more graphic features.
- Kernel size 5 have better performance than 3 because it has wider receptive field.
- 0.001 learning rate has better performance than 0.00001 because it is more likely to find global minima while use the learning rate reducer.

Results

Table

rank	estimators	max depth	min samples split	valid accuracy	runtime
1	500	50	2	0.704406	504.17s
2	500	50	5	0.703781	490.09s
3	250	50	2	0.703125	253.46s
4	500	25	2	0.702875	495.33s
5	250	50	5	0.702719	247.35s
6	250	25	2	0.702281	288.59s
7	500	25	5	0.701875	486.34s
8	250	25	5	0.701875	243.38s

Table 1. Random Forest Results

rank	hidden layer	activation	learning rate	valid accuracy	Runtime
1	256, 128, 64	Relu	0.001	0.701094	258.3s
2	256, 128, 64	Relu	0.0001	0.694531	256.5s
3	128, 64, 32	Relu	0.001	0.691875	244.5s
4	128, 64, 32	Relu	0.0001	0.669531	243.2s
5	256, 128, 64	Sigmoid	0.001	0.667656	288.2s
6	128, 64, 32	Sigmoid	0.001	0.658437	284.0s
7	256, 128, 64	Sigmoid	0.0001	0.555313	293.8s
8	128, 64, 32	Sigmoid	0.0001	0.519531	294.5s

Table 2. MLP Results

rank	filter	kernel size	learning rate	valid accuracy	Runtime
1	128, 256	5	0.001	0.982031	1025.9s
2	128, 256	3	0.001	0.981094	640.8s
3	64, 128	5	0.001	0.977344	436.0s
4	64, 128	3	0.001	0.974688	344.9s
5	128, 256	5	0.0001	0.960312	1087.2s
6	128, 256	3	0.0001	0.948906	733.6s
7	64, 128	5	0.0001	0.934844	456.7s
8	64, 128	3	0.0001	0.929375	376.1s

Table 3. CNN Results

rank	Type	Runtime	test accuracy
1	CNN	1025.9s	0.981375
2	Random Forest	504.17s	0.704406
3	MLP	258.3s	0.706250

Table 4. Best Results

Analysis

Trend

- **Performance:** CNN has a high prediction accuracy which is greater than 98% while the other two are acceptable. The result conforms to theoretical expectations
- **Overfitting:** There is a bit of overfitting on MLP with relu and a high start learning rate. Overall training did not show serious overfitting. A large dataset and suitable preprocessing help improved generalization. The performance of MLP and CNN exceeded theoretical expectations.
- **Runtime:** The training time of MLP and CNN speed up a lot because they can compute on GPU. Random forest can only run on CPU and requires PCA to reduce dimension. Due to hardware limitations, the operating speed differs significantly from theoretical expectations.
- **Number of params:** The number of parameters of random forest influence by the number of trees and their depth. The number of parameters of neuron networks influences by the structure of the model. In addition to theoretical expectations, they also rely on human design.
- **Interpretability:** Since the input data is a pixel image, one-dimensional pixels cannot represent graphic features, random forest and MLP have bad Interpretability. CNN can visualize the kernel.

Exploration

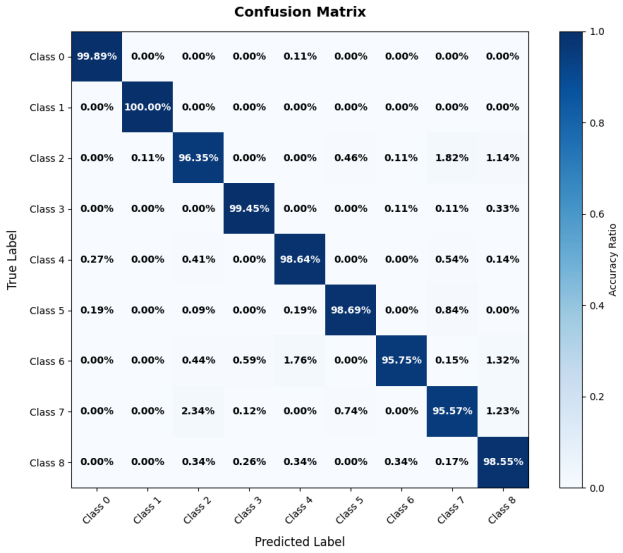

Figure 9. Confusion Matrix For Best CNN

class	precision	recall	f1 score
0	0.9954	0.9989	0.9971
1	0.9988	1.0000	0.9994
2	0.9657	0.9635	0.9646
3	0.9913	0.9945	0.9929
4	0.9745	0.9864	0.9804
5	0.9906	0.9869	0.9888
6	0.9909	0.9575	0.9739
7	0.9593	0.9557	0.9575
8	0.9723	0.9855	0.9788
average	0.9821	0.9810	0.9815

Table 5. Precision, Recall, F1-score Of CNN

The confusion matrix represents the accuracy of the prediction group by class. It represents that all the scores of predictions of all classes are from 95% to 100% which is very high. Besides that, the class 8 that we most care about, have a recall of 98.55%. This results in an extremely low false negative rate for our model. Meanwhile, we also have a high precision for all classes. It represents the misdiagnosis rate for each class is also low.

Conclusion

Main Findings

- CNN has the best performance among all three models. Its high accuracy successfully surpassed human eye recognition. Random forest and MLP's prediction accuracy are too low to replace human identification.
- Although traditional models usually have less computational load, neural networks can train features with less runtime when preserving the complete dimension of images with the GPU acceleration.
- A suitable preprocessing process can greatly improve the generalization ability of the original dataset, and when combined with suitable hyperparameter, it can achieve extremely high accuracy.

Study Limitations

- This project limited us to using traditional models in the sklearn library. They cannot use GPU to speed up.
- Our hardware is the L4 GPU on google colab. It has limited computation ability and RAM to run large batches of high-resolution images.
- We only tried 3 models in this project. There are many other types of models to explore.

Future Work

- For the traditional models, we can explore their variant versions which can speed up by GPU. It will save our training time a lot.
- For hardware, we can run larger batches of image with higher resolution and speed up the training process on more powerful devices.
- For model, we may try more kind of models like transformer reference to frontier research and explore more possibilities.

Reflection

SID 540179745: In this project, I lead the data pre-processing, model building, model training and hyperparameter tuning process. I learned many technics on data preprocessing and CNN model. I found data augmentation especially the rotation operation helps the model to get a higher accuracy by improve model's generalization. I learn interesting model structure such as basic block from ResNet18 [1]. I also use the learning rate reducer tool to make model do detailed training on dataset. Besides that, I also learned how to verify the correctness of hyperparameter combinations using the training curves of neural networks.

SID 550057510: In this assignment, I lead theoretical expectations and analysis. I reviewed the knowledge about the neural network. A neural network is a complex model. There are many layers in the model, I know the effect in each layer. I also learned something about image analysis. I know how to process complex data and build models to analyse it. There are different types in the image. I understand the workflow in this assignment. I also understand the difference between different models. It is suitable in different types.

SID 550460880: Before training the models, I explored the dataset by checking the data shape, pixel range, class distribution, sample images, and brightness differences between classes. This helped me understand why preprocessing is necessary and why different models require different data representations. For example, Random Forest could not directly process image tensors, so I flattened the images, applied standardisation, and used PCA for dimensionality reduction. In contrast, CNNs could directly use the original image structure, which made them more suitable for learning spatial patterns.

SID 541013132: For me, my biggest gain is seeing firsthand how the dimensional shape of the data completely changes what a model actually learns. When we flattened the tissue images into 1D arrays for the Random Forest and MLP, we essentially destroyed the spatial structure of the cells. It became obvious that these models were at risk of learning irrelevant background noise—like the brightness variations caused by the scanning equipment—instead of the actual biological features. Seeing the CNN easily achieve over 98% accuracy simply because its sliding kernels preserved those 2D spatial relationships really made the theoretical concepts from lectures click for me.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.